GESTION DES ÉTUDIANTS
ET DES RÉSULTATS ACADÉMIQUES

Projet de programmation orientée objet (POO) en C++17

Contenu : analyse des classes, relations UML, règles de calcul, code C++ complet et procédure de compilation avec génération automatique d'un graphique.

Important : un PDF est un document de lecture : on ne compile pas directement le PDF. Le fichier **gestion_etudiants.cpp** fourni séparément est le fichier source à compiler. Le PDF contient le même code pour consultation et copie.

Compilation : `g++ -std=c++17 gestion_etudiants.cpp -o gestion`

Exécution sous Windows : `.\gestion.exe`

Après l'exécution, ouvrir **resultats.html** dans Chrome ou Edge pour voir le graphique.

1. Les cinq classes

• **Etudiant :** matricule, nom, niveau, filière et notes. Calcule la moyenne générale et affiche le relevé.

• **Filiere :** code, nom et liste des unités d'enseignement.

• **UE :** code, nom, crédits et liste des matières. Calcule la moyenne de l'UE et sa validation.

• **Matiere :** code, nom et coefficient.

• **Note :** code de la matière et valeur obtenue.

2. Relations et multiplicités UML

Filiere 1 *-- 1..* UE : une filière regroupe une ou plusieurs UE (composition).
UE 1 *-- 1..* Matiere : une UE regroupe une ou plusieurs matières (composition).
Filiere 1 -- 0..* Etudiant : une filière peut accueillir plusieurs étudiants ; chaque étudiant appartient à une filière.
Etudiant 1 -- 0..* Note : un étudiant peut avoir plusieurs notes.
Note * --> 1 Matiere : chaque note concerne une matière.

3. Règles de calcul utilisées

Moyenne d'une UE = somme(note × coefficient) / somme des coefficients des matières notées.
Une UE est validée si sa moyenne est supérieure ou égale à 10/20.
Moyenne générale = somme(moyenne UE × crédits UE) / somme des crédits des UE.

4. Procédure de compilation et affichage du graphique

1. Enregistrer le fichier source sous le nom **gestion_etudiants.cpp**.
2. Ouvrir un terminal dans le dossier contenant le fichier.
3. Compiler avec : `g++ -std=c++17 gestion_etudiants.cpp -o gestion`.
4. Exécuter sous Windows avec : `.\gestion.exe`.
5. Le programme crée **resultats.html** dans le dossier courant. Ouvrir ce fichier avec Chrome ou Edge.

Le graphique est généré à l'exécution, pas pendant la compilation. Il représente la moyenne de chaque UE : bleu si l'UE est validée, rouge sinon.

5. Code source C++17 complet

Le code ci-dessous est également disponible dans le fichier séparé gestion_etudiants.cpp.

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <iomanip>

using namespace std;

// CLASSE MATIERE
class Matiere {
private:
    string code, nom;
    double coefficient;

public:
    Matiere(string c, string n, double coef)
        : code(c), nom(n), coefficient(coef) {}

    string getCode() const { return code; }
    string getNom() const { return nom; }
    double getCoefficient() const { return coefficient; }
};

// CLASSE NOTE
class Note {
private:
    string codeMatiere;
    double valeur;

public:
    Note(string code, double v) : codeMatiere(code), valeur(v) {}

    string getCodeMatiere() const { return codeMatiere; }
    double getValeur() const { return valeur; }
};

// CLASSE UE
class UE {
private:
    string code, nom;
    int credit;
    vector<Matiere> matieres;

public:
    UE(string c, string n, int cr) : code(c), nom(n), credit(cr) {}

    void ajouterMatiere(const Matiere& m) {
        matieres.push_back(m);
    }

    string getNom() const { return nom; }
    int getCredit() const { return credit; }
    const vector<Matiere>& getMatieres() const { return matieres; }

    double calculerMoyenneUE(const vector<Note>& notes) const {
        double somme = 0;
        double totalCoef = 0;

        for (const auto& m : matieres) {
            for (const auto& n : notes) {
                if (m.getCode() == n.getCodeMatiere()) {
                    somme += n.getValeur() * m.getCoefficient();
                    totalCoef += m.getCoefficient();
                    break;
                }
            }
        }

        if (totalCoef == 0) return 0;
        return somme / totalCoef;
    }

    bool estValidee(const vector<Note>& notes) const {
        return calculerMoyenneUE(notes) >= 10;
    }
};

// CLASSE FILIERE
class Filiere {
private:
    string code, nom;
    vector<UE> ues;

public:
    Filiere(string c, string n) : code(c), nom(n) {}
```

```
    void ajouterUE(const UE& ue) {
        ues.push_back(ue);
    }

    string getNom() const { return nom; }
    const vector<UE>& getUEs() const { return ues; }
};

// CLASSE ETUDIANT
class Etudiant {
private:
    string matricule, nom, niveau;
    const Filiere* filiere;
    vector<Note> notes;

public:
    Etudiant(string mat, string n, string niv, const Filiere* f)
        : matricule(mat), nom(n), niveau(niv), filiere(f) {}

    void ajouterNote(const Note& note) {
        notes.push_back(note);
    }

    double calculerMoyenne() const {
        double somme = 0;
        double totalCredits = 0;

        for (const auto& ue : filiere->getUEs()) {
            double moyenneUE = ue.calculerMoyenneUE(notes);
            somme += moyenneUE * ue.getCredit();

            totalCredits += ue.getCredit();
        }

        if (totalCredits == 0) return 0;
        return somme / totalCredits;
    }

    void afficherReleve() const {
        cout << "\n====================================\n";
        cout << "           RELEVE DE NOTES\n";
        cout << "====================================\n";
        cout << "Nom : " << nom << '\n';
        cout << "Matricule : " << matricule << '\n';
        cout << "Niveau : " << niveau << '\n';
        cout << "Filiere : " << filiere->getNom() << '\n';

        cout << "\nDETAIL DES UE\n";

        for (const auto& ue : filiere->getUEs()) {
            double moyenne = ue.calculerMoyenneUE(notes);
            cout << "\nUE : " << ue.getNom() << '\n';

            for (const auto& m : ue.getMatieres()) {
                for (const auto& n : notes) {
                    if (m.getCode() == n.getCodeMatiere()) {
                        cout << "  " << m.getNom() << " : "
                             << n.getValeur() << "/20\n";
                        break;
                    }
                }
            }

            cout << "Moyenne UE : " << fixed << setprecision(2)
                 << moyenne << "/20\n";
            cout << "Credits : " << ue.getCredit() << '\n';
            cout << "Statut : "
                 << (ue.estValidee(notes) ? "VALIDEE" : "NON VALIDEE")
                 << '\n';
        }

        cout << "\nMOYENNE GENERALE : " << fixed << setprecision(2)
             << calculerMoyenne() << "/20\n";
        cout << "====================================\n";
    }

    // Cree une page HTML contenant un graphique SVG.
    // Ouvrir resultats.html dans Chrome ou Edge apres execution.
    void genererGraphique() const {
        ofstream fichier("resultats.html");

        if (!fichier) {
            cout << "Erreur lors de la creation du graphique.\n";
            return;
        }

        fichier << R"HTML(
<!DOCTYPE html>
<html lang="fr">

<head>
<meta charset="UTF-8">
```

```
<title>Graphique des resultats</title>
<style>
body { font-family: Arial; background: #f2f5f9; text-align: center; padding: 25px; }
h1 { color: #17365d; }
svg { background: white; max-width: 850px; width: 95%; border-radius: 12px; }
</style>
</head>
<body>
<h1>Releve graphique des resultats</h1>
<h2>)HTML" << nom << R"HTML(</h2>
<svg viewBox="0 0 850 480" xmlns="http://www.w3.org/2000/svg">
<line x1="90" y1="390" x2="800" y2="390" stroke="black" stroke-width="2"/>
<line x1="90" y1="50" x2="90" y2="390" stroke="black" stroke-width="2"/>
)HTML";

        // Echelle verticale : de 0 a 20.
        for (int i = 0; i <= 20; i += 5) {
            int y = 390 - i * 15;
            fichier << "<text x=\"65\" y=\"" << y + 5
                    << "\" font-size=\"13\">" << i << "</text>\n";
            fichier << "<line x1=\"90\" y1=\"" << y
                    << "\" x2=\"800\" y2=\"" << y
                    << "\" stroke=\"#dddddd\"/>\n";
        }

        int x = 130;
        for (const auto& ue : filiere->getUEs()) {
            double moyenne = ue.calculerMoyenneUE(notes);
            int hauteur = static_cast<int>(moyenne * 15);
            int y = 390 - hauteur;

            fichier << "<rect x=\"" << x << "\" y=\"" << y
                    << "\" width=\"75\" height=\"" << hauteur
                    << "\" fill=\""
                    << (moyenne >= 10 ? "#2196F3" : "#E74C3C")
                    << "\"/>\n";

            fichier << "<text x=\"" << x + 37 << "\" y=\"" << y - 10
                    << "\" text-anchor=\"middle\" font-size=\"14\">"
                    << fixed << setprecision(2) << moyenne
                    << "</text>\n";

            fichier << "<text x=\"" << x + 37 << "\" y=\"415"
                    << "\" text-anchor=\"middle\" font-size=\"12\">"
                    << ue.getNom() << "</text>\n";
            x += 150;
        }

        fichier << R"HTML(
</svg>
<h2>Moyenne generale : )HTML"
                << fixed << setprecision(2) << calculerMoyenne()
                << R"HTML(/20</h2>
<p>Bleu : UE validee | Rouge : UE non validee</p>
</body>
</html>

)HTML";

        fichier.close();
        cout << "\nGraphique genere : resultats.html\n";
    }
};

// PROGRAMME PRINCIPAL
int main() {
    // Creation des UE et de leurs matieres
    UE ue1("UE01", "Mathematiques", 6);
    ue1.ajouterMatiere(Matiere("M01", "Analyse", 3));
    ue1.ajouterMatiere(Matiere("M02", "Algebre", 2));
    ue1.ajouterMatiere(Matiere("M03", "Probabilites", 1));

    UE ue2("UE02", "Informatique", 4);
    ue2.ajouterMatiere(Matiere("M04", "Algorithmique", 2));
    ue2.ajouterMatiere(Matiere("M05", "Programmation C++", 2));

    UE ue3("UE03", "Physique", 2);
    ue3.ajouterMatiere(Matiere("M06", "Mecanique", 1));
    ue3.ajouterMatiere(Matiere("M07", "Electricite", 1));

    // Creation de la filiere
    Filiere filiere("MI", "Mathematiques et Informatique");
    filiere.ajouterUE(ue1);
    filiere.ajouterUE(ue2);
    filiere.ajouterUE(ue3);

    // Creation d'un etudiant
    Etudiant etudiant("MI2026-001", "Jean", "L3", &filiere);

    // Saisie des notes d'exemple
    etudiant.ajouterNote(Note("M01", 14));
    etudiant.ajouterNote(Note("M02", 12));
```

```
    etudiant.ajouterNote(Note("M03", 16));
    etudiant.ajouterNote(Note("M04", 15));
    etudiant.ajouterNote(Note("M05", 13));
    etudiant.ajouterNote(Note("M06", 11));
    etudiant.ajouterNote(Note("M07", 9));

    // Affichage et generation du graphique
    etudiant.afficherReleve();
    etudiant.genererGraphique();

    return 0;
}
```